



Bases de Datos

Programación en ambientes de bases de datos (Procedures, Triggers)

Susana Medina Gordillo
susana.medina@correounivalle.edu.co

PL/SQL: Introducción

—



¿Qué es PL/SQL?



- Lenguaje de procesamiento **procedimental**.
- Implementado por **Oracle**.
- Dispone de estructuras de programación similares a las de la mayoría de los lenguajes de programación.
- Objetivo: Interactuar con la BD.



<https://apexapps.oracle.com/pls/apex/f?p=44785:50:115838704419033::NO::>

Estructura Bloques Anónimos



[**DECLARE**]

Define objetos PL/SQL que serán utilizados dentro del mismo bloque.

BEGIN

Sentencias Ejecutables

[**EXCEPTION**]

Qué hacer si la acción ejecutada causa error

END;

Procedures

—

Procedimientos y funciones



Desde la versión SQL:1999, SQL permite la definición de **funciones, procedimientos y métodos**. Pueden definirse bien mediante el componente procedimental de SQL:1999 o bien mediante un lenguaje de programación externo como Java, C o C++.

Varios sistemas de bases de datos soportan sus propias extensiones procedimentales de SQL, como **PL/SQL** en Oracle y **TransactSQL** en Microsoft SQL Server.

Desde la versión SQL:2003, SQL soporta las funciones que pueden devolver tablas como resultado (**funciones de tabla**).

Estructura Procedimiento



PROCEDURE nombre **IS**

Sección Declarativa

BEGIN

Sección Ejecutable

[**EXCEPTION**]

Sección de Excepciones

END;

Estructura Función



```
FUNCTION nombre RETURN tipo_dato IS
```

Sección Declarativa

```
BEGIN
```

Sección Ejecutable

```
[EXCEPTION]
```

Sección de Excepciones

```
END;
```

Ejemplo: Entidad Bancaria



sucursal (nombre_sucursal, ciudad_sucursal, activos)

cliente (nombre_cliente, calle_cliente, ciudad_cliente)

prestamo (numero_prestamo, nombre_sucursal, importe)

prestatario (nombre_cliente, numero_prestamo)

cuenta (numero_cuenta, nombre_sucursal, saldo)

impositor (nombre_cliente, numero_cuenta)

¿Cómo es la función?



Se necesita una **función** dado el nombre de un **cliente**, devuelva el número de cuentas de las que es titular ese **cliente**.

Solución 1: función

```
SELECT nombre_cliente, calle_cliente,  
ciudad_cliente  
FROM cliente  
WHERE  
recuento_cuentas(nombre_cliente) > 1
```

Solución 1: función cuentas_de

```
select * from table(cuentas_de('Perez'))  
create function cuentas_de (  
nombre_cliente char(20)) returns table (  
    numero_cuenta char(10),  
    nombre_sucursal char(15),  
    saldo numeric(12,2))  
return table  
    (select numero_cuenta, nombre_sucursal,  
    saldo from cuenta where exists  
        (select * from impositor  
        where impositor.nombre_cliente =  
        cuentas_de.nombre_cliente and  
        impositor.numero_cuenta =  
        cuenta.numero_cuenta) )
```

Procedimiento (Procedure)



Un procedimiento es un grupo de **declaraciones PL/SQL** que pueden ser nombradas. Una llamada específica (**call spec**) declara un método de lenguaje de programación como Java o una rutina de lenguajes (3GL) que puede ser llamada por SQL o PL/SQL.

Un **call spec** le dice a la BD Oracle cual método invocar cuando se realice la llamada. También le dice a la BD los tipos de conversiones para los argumentos y el valor a retornar (return).

Solución 2: Procedure



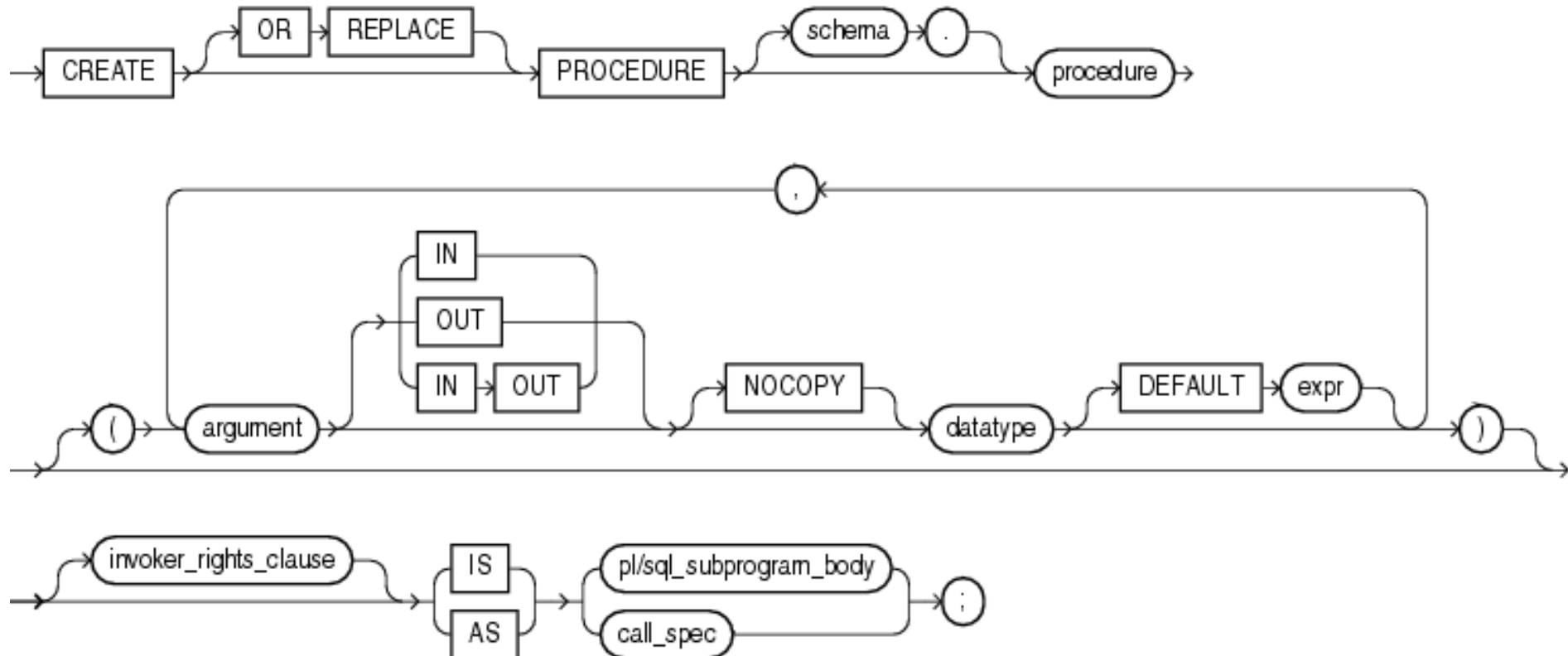
```
CREATE procedure proc_recuento_cuentas
(
    nombre_cliente IN varchar(20),
    recuento_c OUT integer)
BEGIN
    select count(*) into recuento_c
    from impositor
    where impositor.nombre_cliente =
proc_recuento_cuentas.nombre_cliente
END;
```



Invocar Procedure

```
DECLARE recuento_c integer;  
CALL proc_recuento_cuentas('Perez', recuento_c);
```


Procedure Oracle: Create



Sintaxis Procedure Oracle

```
CREATE [ OR REPLACE ] PROCEDURE [ schema.]  
procedure  
    [ (argument [ { IN | OUT | IN OUT } ]  
        [ NOCOPY ]  
        datatype [ DEFAULT expr ]  
    [, argument [ { IN | OUT | IN OUT } ]  
        [ NOCOPY ]  
        datatype [ DEFAULT expr ]  
    ]...  
    )  
]  
[ invoker_rights_clause ]  
{ IS | AS }  
{ pl/sql_subprogram_body | call_spec } ;
```

Procedure Oracle: Ejemplo 1



```
CREATE PROCEDURE remove_emp (employee_id
NUMBER) AS
    tot_emps NUMBER;
BEGIN
    DELETE FROM employees
    WHERE employees.employee_id =
remove_emp.employee_id;
    tot_emps := tot_emps - 1;
END;
```

Procedure Oracle: Ejemplo 2

```
CREATE PROCEDURE get_single_customer (  
    P_ID IN INTEGER,  
    P_AGE OUT INTEGER,  
    P_OCCUPATION OUT VARCHAR2,  
    P_LASTSTATEMENTBALANCE OUT FLOAT )  
BEGIN  
    SELECT AGE, OCCUPATION,  
        LASTSTATEMENTBALANCE INTO P_AGE,  
        P_OCCUPATION, P_LASTSTATEMENTBALANCE  
FROM CROSSSELLCUSTOMERS  
WHERE CROSSSELLCUSTOMERS.ID = P_ID;  
END;
```

Disparadores (TRIGGERS)

—

Disparadores (TRIGGERS)

Los disparadores, llamados reglas **Evento-Condición-Acción**, difieren de las restricciones previamente discutidas en tres formas:

1. Los triggers sólo son probados cuando ocurren ciertos **eventos** específicos (inserciones, borrados, actualizaciones y el fin de una transacción).
2. Un trigger prueba una **condición** y sino ocurre no sucede nada.
3. Si la condición del trigger se cumple, la **acción** definida con el trigger es realizada por el DBMS. La acción (cualquier secuencia de operaciones sobre BD) puede impedir que ocurra el evento, o podría deshacer el evento.

Características Triggers



1. La acción puede ser ejecutada **antes**, **después** o **durante** el evento definido en el trigger.
2. La acción puede referirse a valores viejos o nuevos que se insertaron, borraron o actualizaron en el evento que dispara la acción del trigger.
3. Los eventos de actualización pueden especificar un atributo particular o un conjunto de atributos.
4. Una condición puede ser especificada con una cláusula **WHEN**, y la acción sólo se ejecutará si la regla es activada y se cumple la condición cuando el evento definido en el trigger ocurre.

Características Triggers



5. El programador tiene la opción de **especificar** que la acción sea realizada:

- a) Una vez en cada tupla **modificada**
- b) Una vez en **todas las tuplas que cambian** en una operación de BD

Cuando NO emplear Triggers



En algunas ocasiones es mejor con técnicas alternativas. Por ejemplo: para realizar **resúmenes de datos**. Se empleaban disparadores bajo la **inserción, borrado o actualización** de una relación empleado que contenía los atributos sueldo y departamento para mantener el *sueldo total de cada departamento*. Actualmente, muchos sistemas de bases de datos actuales soportan las **vistas materializadas**.

También se empleaban mucho para las **réplicas de las bases de datos**. Actualmente, hay características específicas para replicar bases de datos.

Vistas materializables



Una vista materializada utiliza el **resultado** de la **consulta** se almacena en una tabla caché real, que será actualizada de forma **periódica** a partir de las **tablas originales**. Esto proporciona un acceso mucho más eficiente, a costa de un incremento en el tamaño de la base de datos y a una posible *falta de sincronía* (los datos de la vista pueden estar potencialmente desfasados con respecto a los reales).

Es una solución muy utilizada en entornos de almacenes de datos (**datawarehousing**).



¿Para qué sirven los Triggers?

- ★ Los triggers sirven para mantener la integridad.
- ★ Propósitos de monitoreo de acceso de usuarios y modificaciones a ciertas tablas.
- ★ Acciones de seguridad sobre tablas .

Disparadores en SQL



Los sistemas de bases de datos basados en SQL emplean mucho los disparadores, aunque antes de **SQL:1999** no eran parte de la norma de SQL. Por desgracia, cada sistema de bases de datos implementó su propia sintaxis para los disparadores, lo que produjo incompatibilidades.

La sintaxis de **SQL:1999** para los disparadores es parecida a la sintaxis del sistema de bases de datos DB2 de IBM y a la de Oracle.

La sintaxis de los triggers varía según el SGBD.

Triggers: Sintaxis SQL Server



```
CREATE [OR ALTER] TRIGGER [schema_name.]
trigger_name
ON { table | view }
[ WITH <dml_trigger_option> [ ,...n ] ]
{ FOR | AFTER | INSTEAD OF }
{ [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE
] }
[ WITH APPEND ]
[ NOT FOR REPLICATION ]
AS { sql_statement [ ; ] [ ,...n ] |
EXTERNAL NAME <method specifier [ ; ] > }
```

Triggers: Sintaxis Oracle



```
CREATE [OR REPLACE] TRIGGER <nombreTrigger>
BEFORE | AFTER                                --Tiempo del evento
{ {INSERT | UPDATE} [OF columna(s)] | DELETE
}
ON <nombreTabla>                            --Eventos
disparadores
FOR EACH ROW | STATEMENT                    --Tipo de Trigger
[WHEN (<condición>) ]                       -- Restricción
Trigger
                                           (solo para triggers
                                           FOR EACH ROW !)
<Bloque PL/SQL>                             --Cuerpo del
```

Triggers: mantenimiento de la integridad



Identifique **operaciones** y **tablas críticas** para la restricción de integridad

Por cada una de tales **tablas** verifique

Si la **restricción** puede ser verificada a nivel de fila entonces

Si la **fila verificada** es modificada en el trigger entonces

Use un **trigger de fila** con **BEFORE**

De lo Contrario Use un **trigger de fila** con **AFTER**

De lo Contrario Use un **trigger de instrucción** con **AFTER**

Ejemplo Trigger

EjecutivosCine (nombre, dirección, codProductor, ingresoNeto)

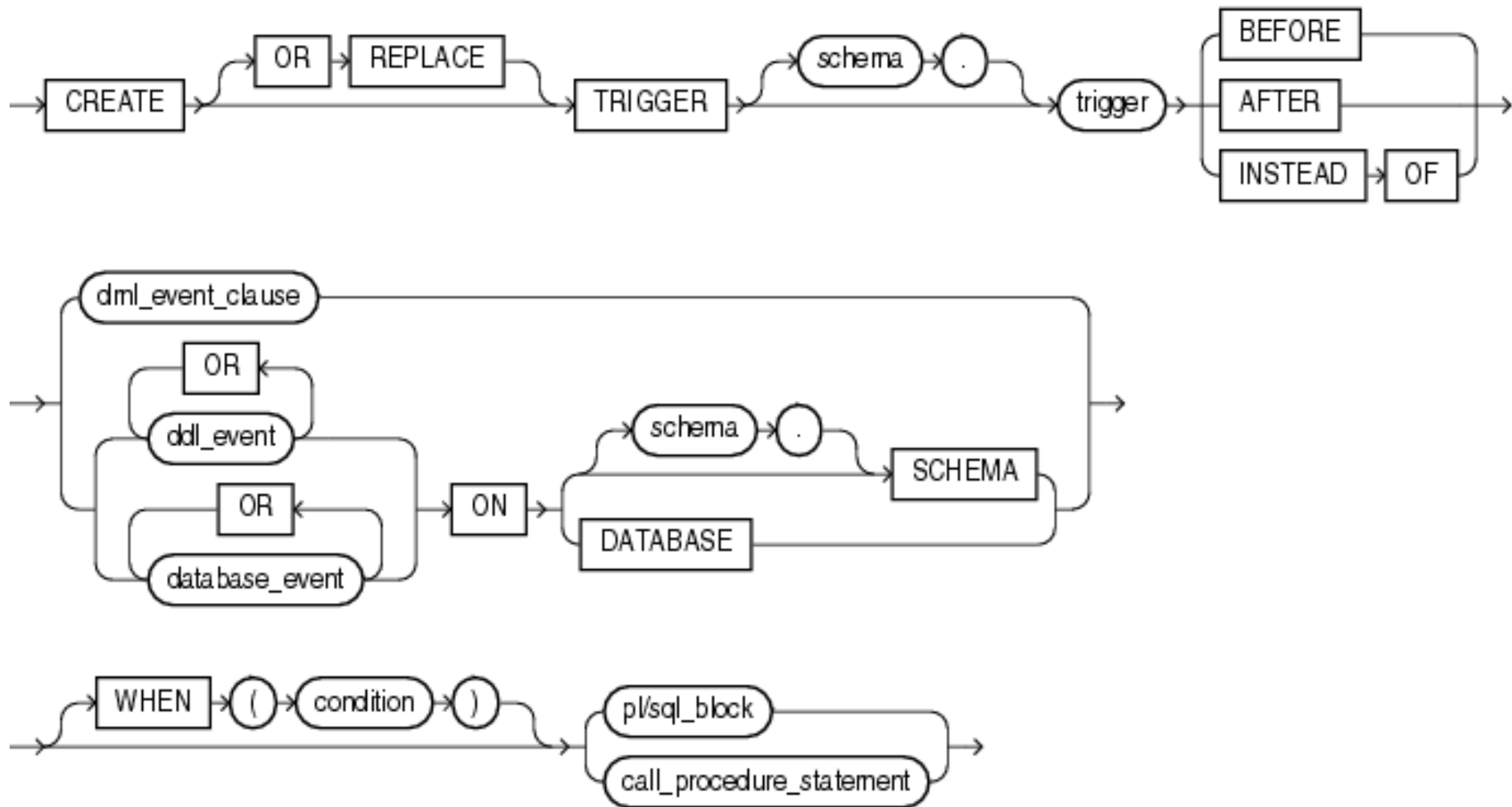
Actualizar al atributo **ingresoNeto**. El efecto de esta regla es restaurar al valor inicial por cualquier intento de reducir el ingreso neto de un ejecutivo:

```
CREATE OR REPLACE TRIGGER ingresoNetoTrigger
  AFTER UPDATE OF ingresoNeto on EjecutivosCine
  REFERENCING
    OLD AS oldTuple
    NEW AS newTuple
  FOR EACH ROW
  WHEN (oldTuple.ingresoNeto > newTuple.ingresoNeto)
  UPDATE EjecutivosCine
  SET ingresoNeto = Oldtuple.ingresoNeto
  WHERE codProductor = Newtuple.codProductor ;
```


Sintaxis Detallada Triggers Oracle

```
CREATE [OR REPLACE] TRIGGER [ schema.]trigger
{ BEFORE | AFTER | INSTEAD OF }
{ dml_event_clause
| { ddl_event [ OR ddl_event ]...
  | database_event [ OR database_event]...
  }
ON { [ schema. ] SCHEMA
      | DATABASE          }
}
[ WHEN (condition) ]
{ pl/sql_block | call_procedure_statement
} ;
```

Triggers Oracle: CREATE



Triggers Oracle: DML_event_clause

```
{ DELETE | INSERT | UPDATE
    [ OF column [, column ]... ]
}

[ OR { DELETE | INSERT | UPDATE
    [ OF column [, column]... ]
}

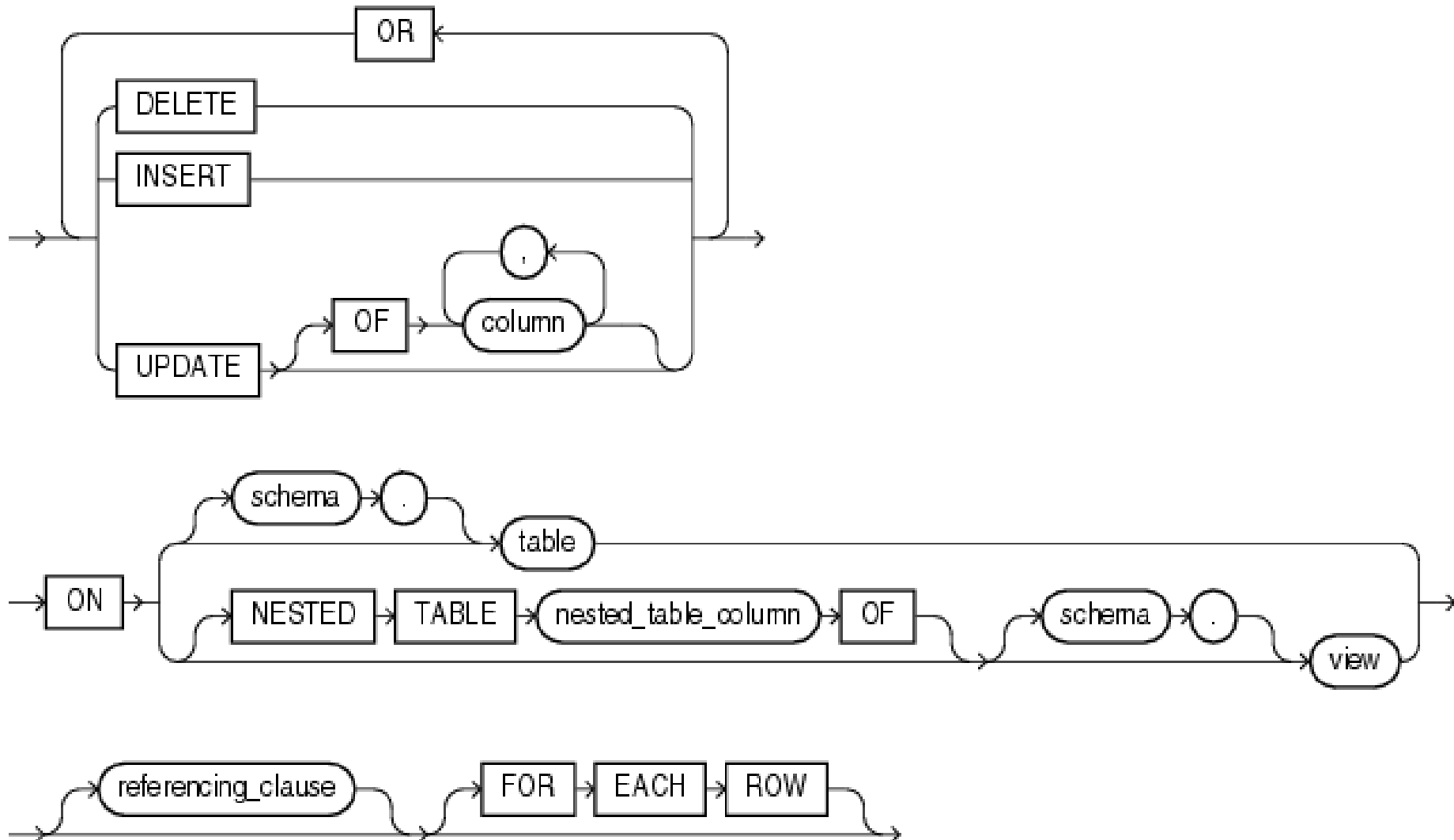
]...

ON { [ schema. ] table
    | [ NESTED TABLE nested_table_column OF ]
    [ schema. ] view
}

[ referencing_clause ]

[ FOR EACH ROW ]
```

Triggers Oracle: dml_event_clause

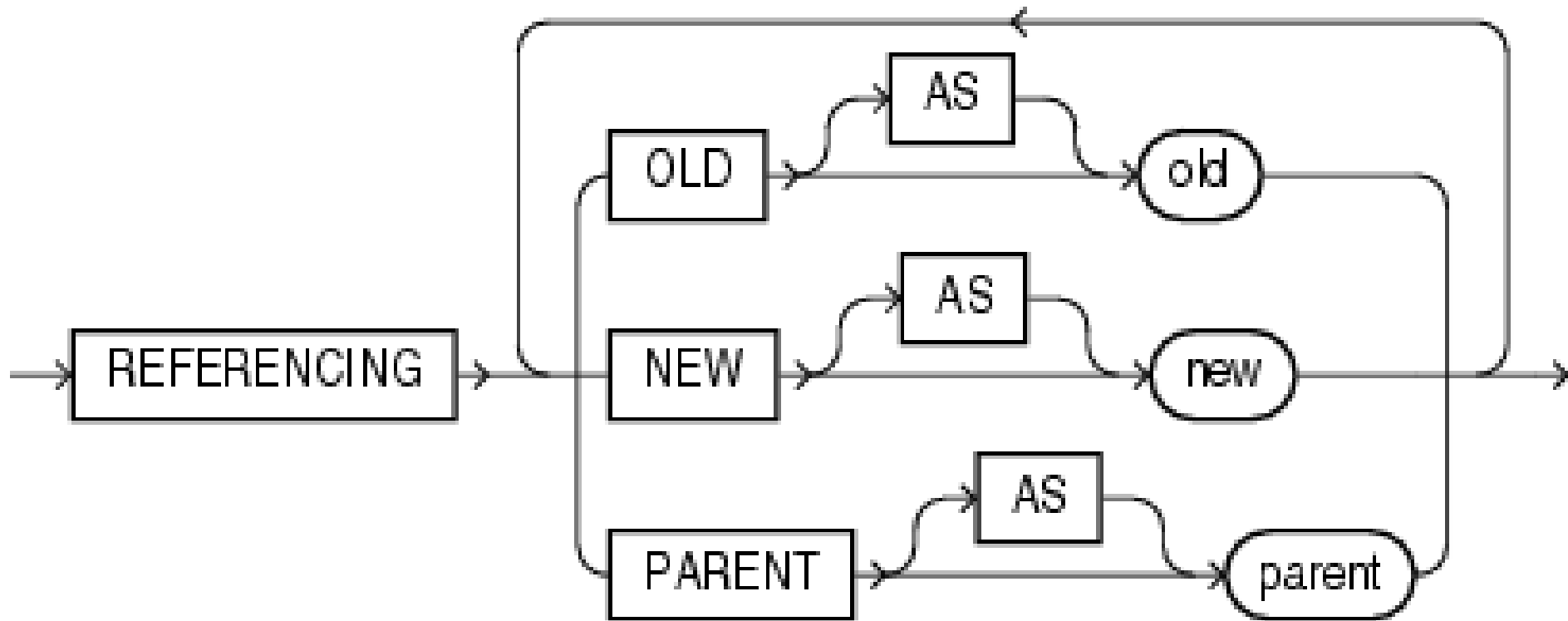


Triggers Oracle: referencing_clause

REFERENCING

```
{ OLD [ AS ] old  
| NEW [ AS ] new  
| PARENT [ AS ] parent }  
  [ OLD [ AS ] old  
  | NEW [ AS ] new  
  | PARENT [ AS ] parent ]...
```

Triggers Oracle: referencing_clause



Diferencias DML, DDL y DCL

DDL (**Data Definition Language**): lenguaje que permite definir estructuras de datos, proporcionado por los SGBD.

Ej: **CREATE, ALTER, DROP**.

DML (**Data Manipulation Language**): lenguaje que utiliza instrucciones de SQL, permite a los usuarios introducir datos para posteriormente realizar tareas de consultas o modificación de los datos que contienen las Bases de Datos.

Ej: **SELECT, INSERT, UPDATE, DELETE**.

DCL (**Data Control Language**): Es un lenguaje con comandos SQL que permiten al Administrador del SGBD, controlar el acceso a los objetos, es decir, otorgar o denegar permisos a uno o más roles. Ej: **GRANT, REVOKE**.

Trigger DML



```
CREATE TRIGGER schema.trigger_name  
    BEFORE  
    DELETE OR INSERT OR UPDATE  
    ON schema.table_name  
    pl/sql_block
```


Ejemplo Trigger DML



```
CREATE TRIGGER hr.salary_check
    BEFORE INSERT OR UPDATE OF salary,
    job_id ON hr.employees
    FOR EACH ROW
    WHEN (new.job_id <> 'AD_VP')
    pl/sql_block
```

Trigger DDL



```
CREATE TRIGGER audit_db_object AFTER  
CREATE  
ON SCHEMA  
pl/sql_block
```

Trigger con llamada a Procedure



```
CREATE TRIGGER salary_check
    BEFORE INSERT OR UPDATE OF salary,
job_id ON employees
    FOR EACH ROW
    WHEN (new.job_id <> 'AD_VP')
    CALL check_sal (:new.job_id, :new.salary,
:new.last_name)
```

Ejemplo Trigger Evento Base de Datos



```
CREATE TRIGGER log_errors AFTER SERVERERROR
ON DATABASE
BEGIN
    IF (IS_SERVERERROR (1017)) THEN
        <special processing of logon error>
    ELSE
        <log error number>
    END IF;
END;
```

Ejemplo Trigger INSTEAD OF (Vistas)

```
CREATE OR REPLACE TRIGGER order_info_insert
  INSTEAD OF INSERT ON order_info
  DECLARE
    duplicate_info EXCEPTION;
    PRAGMA EXCEPTION_INIT (duplicate_info,
-00001);
  BEGIN
    INSERT INTO customers
      (customer_id, cust_last_name,
cust_first_name)
    VALUES (
      :new.customer_id,
      :new.cust_last_name,
      :new.cust_first_name);
```

Ejemplo Trigger INSTEAD OF (Vistas)

```
INSERT INTO orders (order_id, order_date,  
customer_id)  
VALUES (  
    :new.order_id,  
    :new.order_date,  
    :new.customer_id);  
EXCEPTION  
    WHEN duplicate_info THEN  
        RAISE_APPLICATION_ERROR (  
            num=>-20107,  
            msg=>'Duplicate customer or order ID');  
END order_info_insert;
```

/

Comandos Útiles para Triggers



Activar-> `ALTER TRIGGER name ENABLE;`

Desactivar-> `ALTER TRIGGER name DISABLE;`

Información acerca del Trigger->

```
SELECT Trigger_type, Triggering_event,  
Table_name  
FROM USER_TRIGGERS  
WHERE Trigger_name = 'name';
```

Receso 15 min



Práctica en Laboratorio